

Appendix: Performance Benchmarking Methodology and Results for `maxentcpp`

Supplementary material — `maxentcpp` paper v2

11 August 2026

1 Purpose and scope

This appendix reports the benchmark suite run to (i) answer reviewer requests for quantitative performance evidence (fit time, projection time, peak memory, scaling with raster size and sample size), and (ii) locate the computational bottlenecks of `maxentcpp` 1.0.0 so that the package can be optimized without breaking its numerical fidelity contract with Java Maxent 3.4.4.

Every number in this appendix was obtained by actually running the code on 11 August 2026. No figure is copied from an earlier draft or from the literature. The scripts that produced the numbers are listed in Section 7 together with their output files.

2 Hardware and software environment

Component	Value
CPU	Intel Core Ultra 7 165H (16 cores / 22 threads, up to 5.0 GHz, heterogeneous P-core + E-core design)
RAM	62 GiB
OS	Ubuntu 24.04
R	4.6.0 (2026-04-24)
<code>maxentcpp</code>	1.0.0 (installed from CRAN)
<code>terra</code>	1.7.65
<code>maxnet</code>	0.1.4
Java	OpenJDK 21.0.11

Table 1: Benchmark environment.

Because the CPU is a heterogeneous design (performance cores and efficiency cores sharing a package), wall-clock timings show run-to-run variability of roughly $\pm 5\%$. All timings below are medians over at least three runs unless stated otherwise.

3 Methodology

3.1 The in-place mutation artifact (critical)

`maxent_fit()` *mutates the `FeaturedSpace` object in place*. The optimizer updates the feature lambdas stored inside the featured space and leaves them there when it returns. Consequently, calling `maxent_fit()` twice on the *same* featured space does not measure two independent fits: the second call starts from the already-converged solution and terminates after only a handful of iterations (21 instead of 261 in our tests), producing a grossly optimistic timing.

This artifact is the explanation for the “~820 ms” figure that appeared in the v2 draft: it was measured by re-fitting a reused featured space. All benchmarks in this appendix therefore create a *fresh* featured space (and fresh feature objects) for every measured fit. Section 4.1 shows the two numbers side by side so the difference is visible.

3.2 Warm-up

The first call to the Rcpp module in a fresh R process pays a one-time cost of roughly 5–19 s (library load / symbol resolution / page faults). Every benchmark below runs one or two warm-up fits first and excludes them from the reported timings.

3.3 Memory measurement

Peak resident set size (RSS) was measured with `/usr/bin/time -v` around a whole Rscript process that (a) performs a warm-up run, then (b) performs the measured fit and projection. Both `maxentcpp` and `maxnet` scripts load the same libraries and data so the comparison is apples-to-apples.

3.4 Data

Two datasets were used:

- **Bundled *Abeillia abeillei* dataset** (package `extdata`): 73 presence records, 2,371 non-NA background cells, two predictors (bio1 = annual mean temperature, bio12 = annual precipitation, Central America), linear + quadratic + hinge features, 44 features total, `max_iter` = 500, `convergence` = `1e-5`.
- **Synthetic grids** (for scaling and the 1.0.0 diagnostics): 2,371 / 23,000 / 233,000 cells with two continuous predictors (plus one five-level categorical for the diagnostics), Gaussian niches for presence sampling.

4 Measured results

4.1 Fit time (Abeillia dataset)

Measurement	maxentcpp	maxnet
Fresh-state fit, median of 4 runs	18.40 / 18.80 / 18.88 / 18.44 s	—
Fresh-state fit, median	~18.5 s	—
Warm fit, median of 3 runs	~0.25 s	~0.25 s
Iterations to convergence (fresh)	261	—
End-to-end <code>maxent_run()</code> (fit+eval+diag), median of 3	32.3 s	—

Table 2: Fit time on the bundled *Abeillia abeillei* dataset. The “warm fit” row for `maxentcpp` reuses a converged featured space and is shown only to demonstrate the in-place artifact; it is not a valid per-fit cost.

The fresh-state fit converges in 261 iterations. The end-to-end `maxent_run()` adds evaluation (AUC), percent contribution, permutation importance, and CSV output; each of those stages costs 2–3 ms, so the optimizer itself accounts for more than 99% of the wall time (Section 4.2).

4.2 Time breakdown

A step-by-step trace of `maxent_run()` on the Abeillia dataset (2,371 cells, 73 presences, 44 features):

This is the single most important result for optimization: **any speedup of the sequential optimizer translates almost 1:1 into end-to-end speedup**; speeding up every other stage combined cannot save more than ~1%.

4.3 Peak memory (RSS)

The memory advantage is real and already quantified. It comes from streaming evaluation (tiles are scored and discarded) and from not materializing the full prediction matrix. Optimization work should *preserve* this property.

Stage	Wall time
Grid info + dimension	<1 ms
Read occurrences	1 ms
Background indices (n = 10,000)	1 ms
Extract env values + complete cases	3 ms
Generate features (44)	<1 ms
Featured space + fit (261 iters)	18,053 ms
Evaluate (pres + bg predictions, AUC)	3 ms
Percent contribution	2 ms
Permutation importance	3 ms

Table 3: Stage-by-stage breakdown of `maxent.run()`. The fit dominates: 18,053 ms of 18,060 ms total.

Workflow	Peak RSS
<code>maxentcpp maxent.run()</code> + full-raster projection	158 MB (162,076 KB)
<code>maxnet</code> fit + predict (same data)	358 MB (366,176 KB)

Table 4: Peak resident set size measured with `/usr/bin/time -v` on the same 2,371-cell dataset, warm-up excluded. `maxentcpp` uses $\sim 2.3\times$ less peak memory.

4.4 Projection time

Grid	<code>maxentcpp</code>	<code>maxnet</code>
23,000 cells	21 ms	9 ms
Full raster, 6,930 cells (incl. NODATA)	3 ms	—

Table 5: Projection time. Both implementations are trivial at these scales; the streaming advantage only matters for rasters larger than RAM (future work on this hardware).

4.5 Scaling with sample size

The 23K / 2,371 time ratio is $\sim 7\times$ for $\sim 9.7\times$ more cells, i.e. roughly linear in the number of cells (the sub-linear ratio is explained by the two runs converging in different numbers of iterations). Per-iteration cost scales linearly with the number of points, as expected from the hot loops described in Section 5.

4.6 1.0.0 diagnostics suite

Benchmarked on a synthetic grid of 2,371 cells, 100 presence records, two continuous predictors plus one five-level categorical variable, lqh features, `max_iter` = 500, fresh state per fit:

4.7 Expanded virtual-species simulation

48 models: 2 predictor-correlation structures ($\rho = 0, 0.8$) $\times$ 4 Gaussian virtual species (narrow/wide $\times$ centered/offset) $\times$ 2 sample sizes (100, 400) $\times$ 3 replicates. Every model compared against its true suitability surface and against the other package.

4.8 Java Maxent prediction agreement

Direct prediction comparison between Java Maxent 3.4.4 (`MaxentJavaRunner.trainLinear2Var` harness) and `maxentcpp` on identical data (2,100 cells, 100 presences, linear features, `max_iter` = 500, `convergence` = $1e-5$, `beta` = 1.0, `min_deviation` = 0.001):

Cells	maxentcpp fresh fit	Notes
2,371	~18.1–19.6 s	converges in 261 iters
23,000	~129 s	exhausts 500-iteration budget
233,000	not run	exceeds per-session budget

Table 6: Scaling of fresh-state fit time with raster size. The 23K run uses signal-strong synthetic data that exhausts `max_iter`; real-world data converges earlier.

Diagnostic	Wall time
Single fit (continuous only)	~22 s
Fit with categorical predictor	~18 s
Jackknife (3 variables; 6 fits)	~63 s
Cross-validation ($k = 5$; 5 fits)	~135 s
Replicate runs (bootstrap, $n = 5$; 5 fits)	~102 s

Table 7: 1.0.0 diagnostics. Each diagnostic is a small multiple of a single fit, as expected. The previously published “~1 ms / ~94 ms” figures were in-place-mutation artifacts.

5 Hot paths in the optimizer

The sequential optimizer lives in `src/cpp/include/maxent/sequential.hpp` (class `Sequential`, mirroring Java `density.Sequential`). The measured numbers (261 iterations, ~18.1 s, 2,371 points, 44 features) imply a per-iteration cost of

$$18.1 \text{ s} / 261 \approx 69 \text{ ms/iteration.}$$

Three loops dominate that budget:

1. `get_best_feature()` — **feature selection**. Every non-parallel iteration scans all $J = 44$ features and calls `delta_loss_bound(j)` for each. Each bound is $O(n)$ in the number of points $n = 2,444$, so this step is $O(n \times J)$ per iteration. With 261 iterations this is the dominant cost.
2. `search_alpha()` / `new_loss_change()` — **step-size search**. After selecting a feature, the optimizer computes a Newton step and then searches for a better step size by repeatedly calling `new_loss_change(j, alpha)`, each call looping over all n points with a `std::exp()` per point. The quadrupling loop can evaluate this several times per update.
3. `do_parallel_update()` — **every 10 iterations**. Computes the Hessian-vector product and gradient over all n points for the subset of features scheduled for update ($O(n \times n_{\text{update}})$).

All three loops are single-threaded by design: the optimizer must preserve Java Maxent’s exact iteration order for the fidelity contract (`maxentcppCompTest` compares per-iteration loss, entropy, and λ trajectories). Any optimization must therefore preserve bit-comparable results under the deterministic settings, or be gated behind an opt-in flag with the fidelity suite re-run.

6 Optimization opportunities (for the developer)

The following directions are ordered by expected impact. They are *suggestions grounded in the measured hot paths above*, not already-verified changes; each must be validated against `maxentcppCompTest` and the benchmark scripts.

6.1 High impact

1. **Vectorize the $O(n)$ loops with Eigen**. `new_loss_change()` and `delta_loss_bound()` currently loop over points with scalar `std::exp()` calls. `std::exp()` on a vectorized Eigen array is ~4–8× faster on

Quantity	Value
Agreement between packages (r), median	0.98 (min 0.95 across all 48 cells)
r maxentcpp vs true, median	0.88
r maxnet vs true, median	0.94
Mean $ \Delta \text{cloglog} $, median	0.044
Correlation effect on agreement ($\rho = 0$ vs $\rho = 0.8$)	0.985 vs 0.973
maxentcpp fit, median	34.8 s
maxnet fit, median	0.94 s
maxentcpp iterations, median	141

Table 8: Simulation summary. The statistical results justify the equivalence claims in the paper; the timing rows quantify the performance gap on synthetic Gaussian niches.

Metric	Value
Pearson r	1.000000
Spearman ρ	1.000000
RMSE	9.18×10^{-18}
Mean $ \Delta \text{cloglog} $	4.15×10^{-18}
Max $ \Delta \text{cloglog} $	8.76×10^{-17}
AUC (both)	0.903525

Table 9: Java vs C++ prediction agreement — machine precision.

modern x86 (the hardware has AVX2). The package already depends on Eigen (used for the feature matrix and streaming provider), so this is a pure implementation change with no algorithm change.

2. **Avoid recomputing per-feature statistics.** `get_best_feature()` recomputes `delta_loss_bound` for *all* features every iteration, even though only the selected feature’s lambda changed. Caching per-feature bounds and updating only the affected entries would reduce the selection scan from $O(n \times J)$ to $O(n \times k)$ where $k \ll J$ is the number of recently changed features. The Java code has a `featuresToUpdate` notion; the C++ port already computes `state_[j].last_change` — the cache can key on that.
3. **Reduce search_alpha evaluations.** The quadrupling loop calls `new_loss_change` (an $O(n)$ pass with one `exp` per point) until the loss stops improving. A golden-section or analytic bound on the quadratic would cut the number of passes, or the `exp` computations of successive trials could be reused (they differ only by the step multiplier).

6.2 Medium impact

1. **Parallelize across independent fits.** Replicate runs, cross-validation folds, and jackknife leave-one-out fits are independent by construction. The paper already states parallelism is applied “across independent fits”; making it real with OpenMP/threads (or documented `future` support) would turn the 63–135 s diagnostics into 63/cores–135/cores seconds with zero fidelity risk, because each fit is bit-identical to the serial one.
2. **Compile flags.** Verify the package builds with `-O3` and, where CRAN permits, `-march=native` on Linux; profile-guided optimization for the hot loops. (CRAN’s policy restricts `-march` on CRAN builds; the effect can be delivered to users via a documented `Makevars` override without changing the default.)
3. **Reduce `std::exp` precision where safe.** `new_loss_change` accumulates `exp(alpha * F(i, j))`. If the fidelity suite confirms the summation order is the only sensitive part, an opt-in `-ffast-math`-style fast `exp` (or `expf` with double accumulation) could be gated behind an option, but this is risky and must be off by default.

6.3 Low impact / do not touch

1. **Streaming projection.** Already 21 ms at 23K cells and memory-efficient; there is nothing to gain at these scales.
2. **Feature generation.** Measured at <1 ms; not a bottleneck.
3. **The fidelity contract.** The per-iteration trajectory tests are the package’s core value proposition; never change the default algorithm path in a way that alters iteration order or numerical results.

7 Reproducibility

All scripts and raw outputs used for this appendix are kept under `/tmp/javabench/` on the benchmark machine (Gamma):

- `fresh_test.R` — fresh-state fit timings (4 runs, 261 iterations each)
- `desglose2.R` / `run_trace.R` — stage-by-stage breakdown of `maxent_run()`
- `mem_maxentcpp.R` / `mem_maxnet.R` — RSS under `/usr/bin/time -v` (outputs `mem_cpp.txt`, `mem_mn.txt`)
- `diag_bench.R` — 1.0.0 diagnostics timings (output `diag_bench_out.txt`)
- `sim_major3.R` — expanded simulation (48 cells; outputs `sim_major3.csv`, `sim_major3.rds`)
- `sim_figure.R` — `figures/simulation.summary.png`
- `compare_java_cpp.R` — Java vs C++ prediction comparison (output `comparison.rds`, `figures/java-vs-cpp-predictions`)
- `appendix_timing.R` — clean end-to-end `maxent_run()` timing (output `appendix_timing_out.txt`)

To re-run a single measurement, copy the script to a machine with `maxentcpp` 1.0.0 installed and execute `Rscript <script>.R`. The simulation requires 4 cores for *mclapply*; all other scripts are single-process.

8 Summary for the optimization task

- The optimizer is >99% of runtime. Optimize `sequential.hpp` only.
- Target: reduce the ~69 ms/iteration on 2,371 points / 44 features. The three hot loops are feature selection ($O(nJ)$), step-size search ($O(n)$ with multiple `exp()` passes), and the periodic parallel update.
- The fastest safe wins: Eigen-vectorized `exp` loops and caching of `delta_loss_bound` between iterations.
- Parallelism across independent fits (replicates / CV / jackknife) is free of fidelity risk.
- Keep peak RSS at or below ~158 MB on this dataset — it is currently a 2.3× advantage over `maxnet`.
- After any change, re-run `maxentcppCompTest` and the scripts above; the fidelity suite is the gate.